



RAMCO INSTITUTE OF TECHNOLOGY

Approved by AICTE, New Delhi & Affiliated to Anna University
Accredited by NAAC & An ISO 9001:2015 Certified Institution
NBA Accredited UG Programs: CSE, EEE, ECE and MECH

Department of Computer Science and Engineering

Academic Year 2025 – 2026 (Even Semester)

Degree, Semester & Branch: II Semester B.E. CSE 'B'

Course Code & Title: CS25C07 Object Oriented Programming

Name of the Faculty member (s): Dr.B.Vijayalakshmi, ASP-II/CSE

Innovative Practice Description

- **Unit / Topic:** Module 3 / Function and Operator Overloading
- **Course Outcome:** CO3
- **Activity Chosen:** Coding Puzzle
- **Justification:**
Function overloading and operator overloading are important compile-time polymorphism techniques that promote code reusability and modular software design. Students often face difficulty in identifying which overloaded function or operator is invoked during compilation. Coding Puzzle activities encourage learners to explore program behavior, predict outputs, identify errors, and develop logical reasoning skills. By solving coding challenges, students gain practical understanding of inheritance and compile-time polymorphism concepts while improving debugging and analytical abilities.
- **Time Allotted for the Activity:** 15 Minutes
- **Details of the Implementation:**
 - The faculty member introduced the concepts of inheritance, function overloading, operator overloading, and compile-time polymorphism using illustrative examples.
 - Following the lecture, a Coding Puzzle activity was conducted to reinforce the concepts through problem-solving.
 - Students were divided into small teams consisting of 4 members.
 - Each team received one coding puzzle related to function or operator overloading and was instructed to analyze the code, predict the output, identify compilation issues (if any), and propose corrections.
 - Each team discussed the assigned puzzle and documented their observations. Sample puzzle solved is shown in Fig.1.
 - Students collaboratively identified the compile-time decisions taken by the compiler and explained the role of overloaded functions and operators.
 - Selected teams presented their solutions and reasoning to the entire class.
 - The faculty member reviewed all solutions, explained compiler behavior, and highlighted best practices for implementing compile-time polymorphism.

• CO – PO / PSO mapping:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO3	3	3	3	1	1	-	-	1	1	1	1	3	3	3

(1 – Low 2 – Moderate 3 – High)

• PO / PSO mapped:

Innovative practice	PO1	PO2	PO3	PO8	PO9	PO10	PSO1
	3	3	3	1	1	1	3
Justification for correlation	Students apply object-oriented programming principles and compile-time polymorphism concepts to solve coding challenges.	Students analyze program statements and determine compiler behavior for overloaded functions and operators.	Students formulate solutions and modify code to eliminate ambiguities and achieve desired functionality.	Students follow ethical programming practices and coding standards while designing solutions.	Students work effectively in teams to solve puzzles and discuss alternative approaches.	Students communicate their interpretations and coding solutions during class presentations	Students demonstrate software development skills through object-oriented programming exercises.

• Images / Screenshot of the practice:

RAMCO INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Academic Year: 2025- 2026 (Even Semester)
Module 3 Coding Puzzle activity- OOPS

R. Deepika
S. Lakshmi Priya

Coding Puzzle 1 – Function Overloading Resolution
A program containing overloaded functions:
display(int);
display(float);
display(double);

Challenge: Determine which function will be called for the statements:
display(10); → calls the first function display (int)
display(10.5); → calls display (double) function.
display('A'); → no matching function- if char converted to int then it calls display (int).
Students were asked to justify the compiler's choice.

Fig 1. Sample Coding puzzle with answers

Reflective Critique:

❖ *Feedback of practice from students and other stakeholders:*

- Students reported improved confidence in understanding function overloading and operator overloading concepts.
- Students expressed that predicting outputs before execution enhanced their logical thinking and debugging skills.

❖ **Benefit of the practice:** (E.g.: Outcome attainment would have increased due to innovative practice over conventional practice)

- Improved conceptual understanding of compile-time polymorphism.
- Enhanced analytical and debugging abilities.
- Increased confidence in implementing overloaded functions and operators.

❖ **Challenges faced in implementation:**

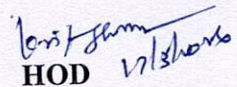
- Some students initially struggled to understand overload resolution rules.
- A few teams required additional guidance when dealing with ambiguous function calls.
- Managing time effectively while discussing multiple coding puzzles was challenging.

References:

- <https://www.readingrockets.org/classroom/classroom-strategies>

 17/8/2026

Signature of Faculty Member


HOD 17/8/2026